Recitation 3

Lecture overview

- Pattern search – (`grep`)
- Text substitution – search and replace (`sed`)
- File Difference – (`diff`)

Regular expressions

- Regular expressions (regexp) are useful constructs describing family of strings (patterns).
- Several UNIX/Linux text utilities use regexp.
- We will learn two regexp-powered commands:
  - `grep` – searching for a specific pattern
  - `sed` – performing a search and replace
- We will discuss several theoretical concepts related to regexp:
  - Connection to finite state machines
  - Matching strategies

`grep` – searching for matching patterns

- The `grep` command searches files for patterns defined by regular expressions, and **prints matching lines**

```
grep [options] regexp [files]
```

- `grep` = **g**lobally search a **r**egular **e**xpression and **p**rint

Searching for a specific string

The simplest regular expression is just a sequence of characters, designating an identity match to that string

- Example:

```
grep Peter phone_dir.txt
```

```
BENNETT, Peter 7456
HARMER, Peter 7484
MAKORTOFF, Peter 7328
```

Task 1

- Print lines satisfying the following requirements from `phone_dir.txt`:
- `grep-ex1.txt` – last name ends with a vowel (A,E,I,O,U)

```
grep '[AEIOU],' phone_dir.txt
```

- `grep-ex2.txt` – 1st letter of the last name is not B and 2nd letter is a vowel

```
grep '^[^B][AEIOU]' phone_dir.txt
 - or -
grep -v '^B' phone_dir.txt | grep '^.[AEIOU]'
```

- `grep-ex3.txt` – first name has at least 5 letters and 3rd digit in number is 5

```
grep -E '[A-Z][a-z]{4,} ..5.$' phone_dir.txt
```

Assume each line has the format:

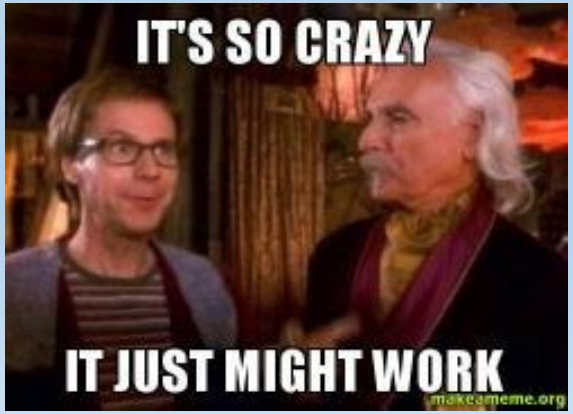
<LAST_UC>,<SPACE><FIRST_LC><SPACE><4DIGIT_NUM>

Task 2

- Write a piped sequence of commands that lists all the numbers written in **a-tale-of-two-cities** into **story-numbers.txt** (a new file which you'll create)
- For this purpose, a number is defined as a sequence of digits that:
  - Does not start with a leading 0
  - Immediately follows a white space or a punctuation mark
  - Has a white space or a punctuation character immediately after it
- For example, the following line contains exactly **four** numbers (highlighted in bold red):September 12th,**1906**. . . I worked 08.**30**-**16**.**30**,.
- Each number in the new file should appear once and sorted according to the numeric order.

Task 2 – strategy

- Preparation of input:
  - We would like to translate all punctuation marks and spaces to new lines.
  - Then we would like to “get rid” of the blank lines that will appear.
- Print relevant items
  - We would like to “grab”(grep) all sequences of numbers that don’t start with a 0.



Task 2 – solution

```
cat a-tale-of-two-cities-full.txt | \
tr [:space:][:punct:] "\n" | \
tr -s "\n" | \
grep "^[1-9][0-9]*$" | \
Sort -n | \
uniq > story-numbers.txt
```

Importance of the $ symbol in this solution

The $ stands for the end of the line.
Meaning we want our text to end with a number.

If we skip the $ we can get results such as:
1st, 10th, 30cm.

sed – stream editor

- **sed** is a **s**tream **ed**itor for text, which can perform transformations on an input stream, based on regular expressions and simple instructions
- **sed** has several command forms. We will focus on the substitute command ('s') applied in the following format:

```
sed 's/pattern/replacement/[g]' [file]
```

sed – stream editor

```
sed [options] 's/pattern/replacement/' [file]
```

- If a line has a match for **pattern**, the first of the matching strings is replaced by **replacement**
  (more details later for cases with multiple matches)
- If a line does not have a match for **pattern**, it is printed as is
- Modified version of **file** is printed by default to **stdout**
- File can be directly edited by using the **-i** option.
  Useful, but risky!
- We will use the **-r** option for extended regular expressions.

sed – examples

phone_dir.txt:

```
ADAMS, Andrew 7583
BARRETT, Bruce 6466
. . .
```

- Change `','` delimiter to `';'`:

```
sed -r 's/,/;/' phone_dir.txt | head -n1
```

```
ADAMS; Andrew 7583
```

- Remove `','` delimiter:

```
sed -r 's/,//' phone_dir.txt | head -n1
```

```
ADAMS Andrew 7583
```

- Mask phone number with ##:

```
sed -r 's/[[:digit:]]+$/##/' phone_dir.txt | head -n1
```

```
ADAMS, Andrew ##
```

Task 3

Print the largest number in the story using the following message: The largest number in the story is <num>

Use pipe from task 2 and an additional sed command.

Task 3

The file is sorted therefore the largest number can be obtained by using the tail command (tail –n1)

Solution:

```
tail –n1 story-numbers.txt | \
sed –r 's/.*/The largest number in the story is &./'
```

Alternative solution 1:

```
echo "The largest number in the story is "` tail –n1
story-numbers.txt` "."
```

Alternative solution 2:

```
largest_number=`tail -n1 story-numbers.txt`
echo "The largest number in the story is $largest_number."
```

The `diff` command

The `diff` command compares two text files.

```
==> cat file1
```

```
bone
two
three
four
```

```
==> cat file2
```

```
one
two
four
five
```

line change

content in file1

content in file2

line deletion

line addition

```
==> diff file1 file2
1c1
< bone
---
> one
3d2
< three
4a4
> five
```

```
diff symbols are:

a: add
c: change
d: delete
```

```
Line at left file
Line at right side
```

Use diff to compare your results against files in `/share/classes/class03/`